

Multi-Dimensional Personalized Benford Conformance for Credit Card Fraud Detection: A Negative Result and Its Mechanistic Explanation

Ivory Le
Lehigh University, MATH341

May 2026

Abstract

Benford’s Law is widely used in forensic accounting to detect financial statement fraud: legitimate financial data is expected to follow a logarithmic leading-digit distribution, while fabricated numbers are expected to deviate. Recent work [8] extended this idea to credit-card fraud detection, reporting that Benford features improve machine-learning classifiers on the European Credit Card Fraud dataset. We test whether this result transfers to the more interpretable IEEE-CIS Fraud Detection dataset using a per-user, multi-dimensional Benford analysis that measures conformance separately for transaction amounts, inter-transaction time deltas, and amount ratios. We introduce an *Excess MAD* correction derived from Monte Carlo simulation to handle small per-user samples. Contrary to the hypothesis, we find that across all five product categories in IEEE-CIS, legitimate transactions deviate from Benford’s Law *more* than fraudulent transactions. A Random Forest classifier using our per-user Benford features does not improve over a baseline without them (AUC 0.749 baseline vs. 0.739 with Benford). We attribute the inversion to two mechanisms: (i) subscription-style recurring charges at psychologically-chosen price points inflating legitimate users’ digit deviation, and (ii) automated card-testing probes by fraudsters sampling varied amounts that incidentally approximate Benford’s distribution. This boundary condition has implications for the deployment of Benford-based fraud detection in real e-commerce settings.

1 Introduction

Credit-card fraud represents a persistent and costly threat to consumers, merchants, and payment networks. Modern fraud-detection systems combine rule-based heuristics with machine learning trained on historical transaction data. A recurring theme in the forensic-accounting literature is that fabricated numbers tend to violate *Benford’s Law*, which specifies that in many naturally-occurring datasets the probability of a leading digit $d \in \{1, \dots, 9\}$ is

$$\Pr[D_1 = d] = \log_{10} \left(1 + \frac{1}{d} \right). \quad (1)$$

This distribution was empirically observed by Newcomb [1] and Benford [2], rigorously justified as the unique base-invariant law on significant digits by Hill [3], and operationalized for auditing by Nigrini [6] and Durtschi et al. [7].

A recent undergraduate study by Fu [8] applied this intuition to credit-card fraud using the European Credit Card Fraud dataset and reported that fraudulent transactions deviate from Benford’s Law more than legitimate ones, validating Benford features as a useful signal for machine-learning classifiers. The European dataset, however, is anonymized through Principal Component Analysis (PCA), which obscures the underlying transaction features and makes it impossible to investigate *why* the signal works, or whether it generalizes.

This project had two starting goals. First, extend the ODU approach from aggregate to *per-user* Benford analysis, computing conformance individually for each cardholder rather than across the entire dataset. Second, extend from single-dimensional (amounts only) to *multi-dimensional* Benford analysis that also tests inter-transaction time deltas and amount ratios. We deliberately use the IEEE-CIS Fraud Detection dataset [9] rather than the European dataset, because IEEE-CIS preserves interpretable features (card fingerprints, product categories, email domains), enabling mechanistic investigation of whether and why Benford signals hold.

Our result is a *negative* one: the expected fraud-detection signal inverts on IEEE-CIS. Legitimate transactions deviate from Benford’s Law more than fraudulent transactions in every product category, and per-user Benford features do not improve Random Forest fraud classification. The interpretable-feature setting allows us to identify two complementary mechanisms driving this inversion: subscription-dominated legitimate user behavior, and automated card-testing patterns in fraud. This suggests that the Benford-for-fraud paradigm, effective for aggregate financial-statement analysis, has a scope limitation that becomes visible only when analysis descends to the per-user level on realistic retail data.

2 Methods

2.1 Benford’s Law and Its Derivation

Equation (1) can be derived from the principle of scale invariance [5]: if a law governs the leading digits of natural data, that law should be independent of the unit of measurement. Let $P(x)$ denote the probability density of a positive random variable X . Scale invariance requires $P(\lambda x) d(\lambda x) = P(x) dx$ for all $\lambda > 0$, which simplifies to $\lambda P(\lambda x) = P(x)$. Differentiating with respect to λ at $\lambda = 1$ yields the functional equation $xP'(x) + P(x) = 0$, whose solution is $P(x) = C/x$. Under this density, $Y = \log_{10}(X)$ is uniformly distributed, and the fractional part $\{Y\}$ is uniform on $[0, 1)$. Since $D_1 = d$ if and only if $\log_{10}(d) \leq \{Y\} < \log_{10}(d+1)$, integrating the uniform density over this interval gives (1).

2.2 Why Benford Should Apply to Transaction Data

We test three derived quantities per user; each requires a theoretical justification for why Benford’s Law should hold.

Transaction amounts. Financial data combine many sub-populations (groceries, fuel, rent, travel, utilities, subscriptions), each with its own generating distribution and characteristic scale. By Hill’s mixture theorem [3], unbiased mixtures of randomly-chosen distributions converge to Benford’s Law.

Inter-transaction time deltas. Waiting times between transactions on a single account are typically heavy-tailed, spanning seconds (rapid online purchases) to weeks (rent payments). Engel and Leuenberger [4] showed that exponentially-distributed data approximate Benford’s Law closely.

Amount ratios. Let $r_i = x_i/\bar{x}_u$ denote transaction i normalized by user u ’s mean amount. If amounts are approximately log-normal (an empirical regularity of financial data), ratios of log-normals are themselves log-normal and conform approximately to Benford.

The central hypothesis was: fraudulent activity disrupts one or more of these dimensions in detectable ways. This hypothesis was falsified empirically; see Section 3.

2.3 Goodness-of-Fit Tests

Given counts O_d of leading digits $d = 1, \dots, 9$ from N observations and expected counts $E_d = N \log_{10}(1 + 1/d)$, we use the Mean Absolute Deviation:

$$\text{MAD} = \frac{1}{9} \sum_{d=1}^9 \left| \frac{O_d}{N} - \log_{10} \left(1 + \frac{1}{d} \right) \right|. \quad (2)$$

MAD is preferred over Pearson chi-squared in the forensic accounting literature because chi-squared develops excess statistical power at large N : real datasets rarely follow Benford’s Law exactly, so large samples cause small systematic deviations to register as significant [6, 7].

2.4 Excess MAD: Small-Sample Correction

When applied per-user, MAD is biased upward because individual users have few transactions. Even under a perfect Benford null, finite-sample fluctuations inflate MAD. To correct for this, we estimate the expected MAD under the null via Monte Carlo simulation:

$$\text{MAD}_{\text{excess}} = \text{MAD}_{\text{observed}} - E[\text{MAD}(N)]. \quad (3)$$

We simulate 5,000 trials per sample size across $N \in \{10, 15, 20, \dots, 1000\}$ by drawing digits from the true Benford distribution and computing MAD. As shown in Figure 1, the empirical $E[\text{MAD}(N)]$ follows the theoretical scaling $E[\text{MAD}] \approx k/\sqrt{N}$ with $k \approx 0.235$. This correction is essential: at $N = 30$, the expected MAD under the Benford null is already 0.043, more than seven times Nigrini’s “close conformity” threshold of 0.006. Without correction, low-volume users would falsely appear non-conforming regardless of behavior.

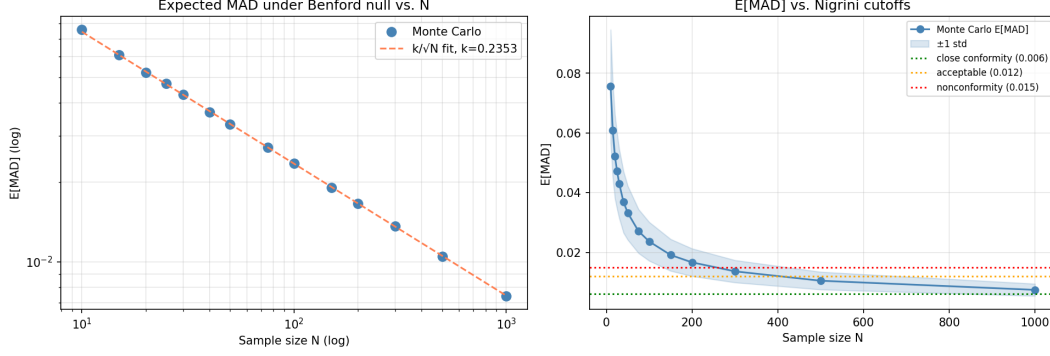


Figure 1: Monte Carlo estimate of expected MAD under the Benford null as a function of sample size N , across 5,000 trials per N . Left panel (log-log scale) shows the tight $k/\sqrt{N}$ fit with $k = 0.235$. Right panel shows the absolute magnitude relative to Nigrini’s empirical conformity thresholds.

2.5 Per-User Multi-Dimensional Features

For each user u with at least 30 transactions, we compute a three-dimensional Benford fingerprint:

$$s_u^{\text{amt}} = \text{MAD}_{\text{excess}}(\{x_i : i \in u\}), \quad (4)$$

$$s_u^{\text{time}} = \text{MAD}_{\text{excess}}(\{\Delta t_i : i \in u\}), \quad (5)$$

$$s_u^{\text{ratio}} = \text{MAD}_{\text{excess}}(\{x_i/\bar{x}_u : i \in u\}). \quad (6)$$

The threshold $N \geq 30$ is a standard practical minimum for reliable Benford analysis [7].

2.6 Dataset and User Inference

The IEEE-CIS Fraud Detection dataset [9] comprises 590,540 labeled transactions from Vesta Corporation, a payments processor, with a fraud rate of 3.50%. Transaction amounts span from \$0.25 to \$31,937 (5.1 orders of magnitude), satisfying the multi-decade scale requirement of Benford’s Law. We exclude the auxiliary identity file (`train_identity.csv`) because its features are primarily categorical encodings and bounded integer codes that do not satisfy Benford’s scale requirement.

IEEE-CIS does not provide an explicit user identifier, a realistic condition for payment-processor data. Following Kaggle community practice, we infer user identity from the triple (`card1`, `addr1`, `P_emaildomain`). This yielded 90,375 inferred users, of which 2,953 (3.3%) had ≥ 30 transactions and are therefore analyzed. Of these frequent users, 1,165 (39.5%) had at least one fraudulent transaction.

2.7 Machine Learning Protocol

To test whether per-user Benford features improve fraud detection end-to-end, we train Random Forest classifiers (200 trees, max depth 10, class-weight balanced) under three feature configurations:

1. **Baseline:** transaction-count, mean amount, standard deviation, median amount, maximum amount, median time delta.
2. **ODU-style:** baseline + aggregate per-user raw MAD on amounts (single-dimensional, no small-sample correction).
3. **Ours:** baseline + three-dimensional Excess MAD features from Section 2.5.

We split at the user level (75% train, 25% test, stratified by fraud label, seed = 42) and report AUC-ROC, precision, recall, and F1 on the held-out users.

3 Results

3.1 Validation on Synthetic Data

Before evaluating on IEEE-CIS, we validated the pipeline on a synthetic dataset where legitimate amounts follow a log-normal distribution (mixture of sources) and fraudulent amounts cluster around round numbers (\$50, \$100, \$500, \$1000, \$2000). As expected, legitimate transactions conformed tightly to Benford’s Law (MAD = 0.0007), while fraudulent amounts deviated substantially (MAD = 0.0266), confirming that the method detects the hypothesized pattern when the data match the theoretical assumptions.

3.2 Inversion on Real Data

The per-user feature analysis on IEEE-CIS showed the opposite of the synthetic result. Table 1 reports mean Excess MAD by class across the 2,953 frequent users. Legitimate users show higher Excess MAD than fraudulent users on both the amount and ratio dimensions. The time-delta dimension is near zero for both classes, indicating no signal in that dimension on this dataset.

Table 1: Per-user Excess MAD by class on IEEE-CIS. Legitimate users deviate from Benford more than fraudulent users on amount and ratio dimensions.

Feature	Legitimate (mean)	Fraud (mean)	Difference
<code>benford_amt</code>	0.0283	0.0219	−0.0064
<code>benford_time</code>	0.0026	0.0033	+0.0007
<code>benford_ratio</code>	0.0320	0.0259	−0.0061

3.3 Per-Category Analysis

Table 2 shows the same pattern holds across all five product categories in IEEE-CIS. In every category, legitimate transactions deviate from Benford more than fraudulent transactions. Additionally, legitimate MAD varies substantially across categories (from 0.018 for category C to 0.071 for category R, a $4\times$ range), suggesting that IEEE-CIS does not satisfy Hill’s mixture-theorem assumptions at the category level: the data within each category

is constrained by merchant-specific pricing regularities rather than an unbiased mixture of distributions.

Table 2: Benford MAD by ProductCD. In every category, legitimate transactions deviate more than fraudulent ones (negative “diff” values), contradicting the expected direction of the fraud signal.

Category	n_{legit}	n_{fraud}	Legit MAD	Fraud MAD	Diff
C	60,511	8,008	0.0176	0.0130	−0.0046
H	31,450	1,574	0.0696	0.0659	−0.0037
R	36,273	1,426	0.0711	0.0621	−0.0090
S	10,942	686	0.0550	0.0446	−0.0104
W	430,701	8,969	0.0312	0.0275	−0.0036

3.4 Machine-Learning Evaluation

Even if the signal direction is inverted, a Random Forest classifier can in principle learn to use it. Table 3 compares the three feature configurations. The baseline model without any Benford features achieves the highest AUC (0.749). Adding the ODU-style aggregate feature reduces AUC by 0.004; adding our three-dimensional Excess MAD features reduces AUC by 0.010. Precision, recall, and F1 show the same directional pattern. Figures 2 and 3 visualize the result: the three ROC curves are nearly overlapping, with baseline slightly on top.

Table 3: Random Forest performance on held-out test set (25% stratified split, 739 users, 292 fraud-positive). Adding Benford features does not improve classification and slightly hurts AUC.

Model	AUC-ROC	Precision	Recall	F1
Baseline	0.7490	0.6721	0.5616	0.6119
ODU-style	0.7453	0.6824	0.5445	0.6057
Ours (3D Benford)	0.7392	0.6597	0.5377	0.5925

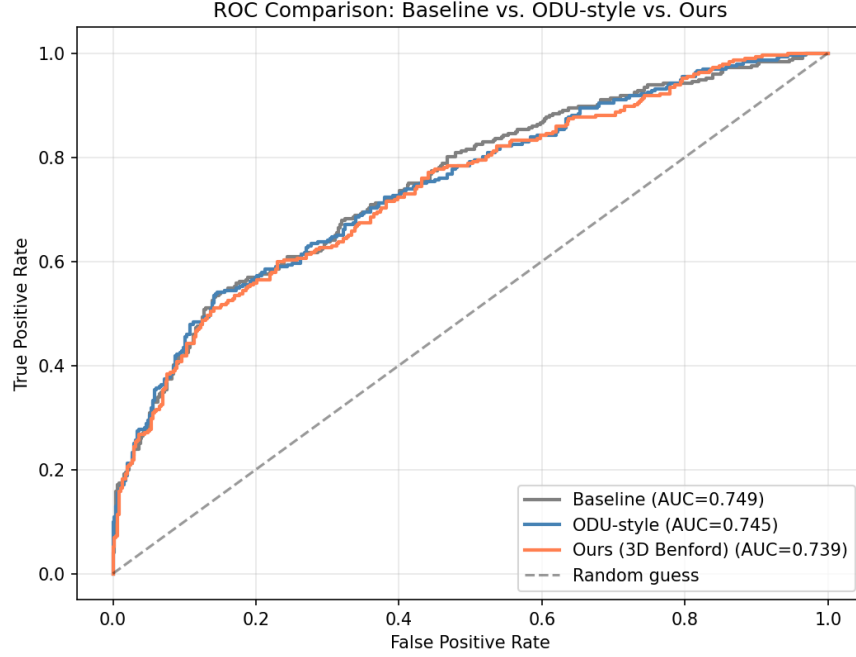


Figure 2: ROC curves overlap closely across all three feature configurations.

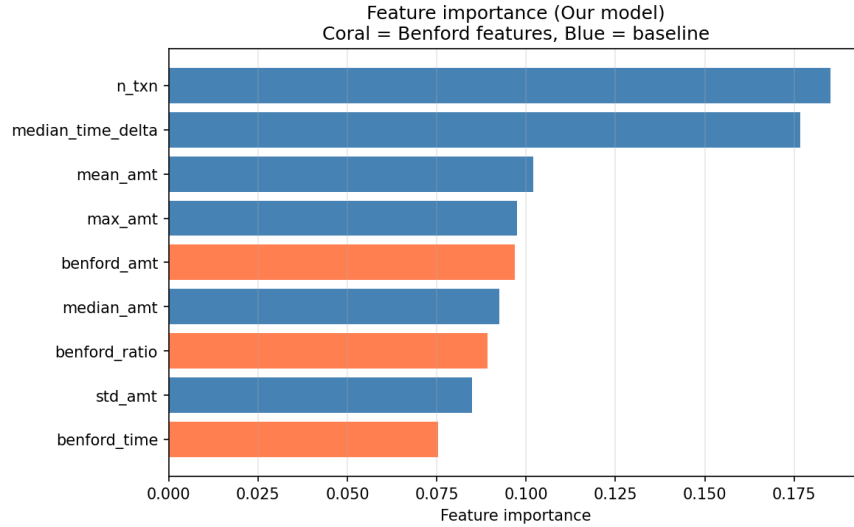


Figure 3: Feature importance for the “Ours” model. Benford features (coral) are used by the Random Forest but this does not translate to improved generalization performance.

The feature-importance plot reveals a subtle but important point: the Random Forest *does* use the Benford features (they contribute roughly 26% of total split importance), yet this does not translate to improved AUC on held-out users. The features help the model fit the training set but do not generalize, indicating the per-user Benford signal is too noisy or too entangled with other factors to constitute a robust predictor.

4 Discussion: Why the Signal Inverts

Our central empirical finding – that legitimate transactions deviate from Benford’s Law more than fraudulent transactions in every product category of IEEE-CIS – requires explanation. We identify three complementary mechanisms revealed by direct inspection of user transaction patterns.

4.1 Mechanism 1: Subscription Contamination

Inspection of individual legitimate frequent users reveals heavy repetition of specific dollar amounts. One legitimate user ($n = 45$ transactions) shows \$59.00 appearing 27% of the time, \$30.95 appearing 20% of the time, and \$117.00 appearing 16% of the time – a clear signature of three to four recurring subscriptions. Across our 1,788 legitimate frequent users, 64.9% of transactions carry an amount that appears at least twice for that user.

Recurring subscription prices cluster on a handful of specific digits (typically leading with 1, 2, 5, or 9 because of psychological pricing at \$9.99, \$14.99, \$19.99, \$29.99, \$59.00, \$99.00, etc.). These clusters violate the continuous-scale assumption underlying Benford’s Law: a handful of discrete price points, repeated many times, cannot produce a smooth logarithmic distribution regardless of the underlying population of users.

4.2 Mechanism 2: Card-Testing Probe Distributions

Conversely, the largest fraudulent user we inspected ($n = 1,452$ transactions) had 909 unique amounts spanning \$0.93 to \$301 (2.51 orders of magnitude). Its leading digit distribution was the closest to Benford in our sample ($MAD = 0.017$), with digit 1 appearing 28% of the time and digit 2 appearing 24% of the time – values consistent with Benford’s 30% and 18%.

This pattern is consistent with automated card-testing attacks, in which a script iterates through varied dollar amounts to validate a stolen card’s balance. An attack script enumerating probe amounts from a uniform distribution over log-space would, by construction, approximate Benford’s Law. This is the precise opposite of the hand-fabricated round numbers that Benford was originally designed to detect in financial-statement fraud: machine-generated probe amounts inherit the uniformity of their generating algorithm, which happens to match Benford’s logarithmic structure.

4.3 Mechanism 3: Mixture-Theorem Violation

Hill’s mixture theorem [3], which underlies the theoretical justification for Benford’s Law on financial data, requires that the observed data be an unbiased mixture of distributions spanning a wide range of scales. The IEEE-CIS category-level analysis in Table 2 shows this assumption fails: legitimate MAD varies from 0.018 to 0.071 across the five product categories, indicating that each category imposes its own merchant-specific pricing regularities. Within a single category, users see a constrained price range rather than an unbiased mixture, and this constrains the leading-digit distribution away from Benford. Aggregate Benford analysis across all categories partially recovers the mixture property; per-user and per-category analysis, which strip that mixture away, reveals the underlying merchant-specific distortion.

4.4 Scope of the Conclusion

These findings do not contradict Benford’s Law as a tool for forensic accounting (financial-statement fraud), where the mixture assumption is more plausible and where the fabrication mechanism (humans inventing numbers) produces characteristic biases. Our negative result applies specifically to: (i) per-user analysis, which removes the mixture; (ii) per-transaction retail e-commerce data, where pricing is constrained by marketing; and (iii) automated fraud, where the fabrication mechanism is algorithmic rather than psychological. These three conditions are all simultaneously present in IEEE-CIS, which is also a realistic description of the modern online fraud landscape.

4.5 Implications for Fraud Detection

A practical implication is that Benford-based fraud detection is unlikely to survive the transition from batch financial-statement auditing to real-time per-transaction e-commerce. Fraud detection systems in production settings (e.g., commercial payment processors) therefore should not treat Benford conformance as a standalone signal at the per-user level. The feature importance and AUC results in Section 3.4 show that even when a sophisticated classifier is allowed to use Benford features, performance does not improve – the signal, in its inverted direction, is too noisy to constitute a robust predictor relative to simpler transaction-pattern features.

5 Conclusion

We set out to extend the ODU result [8] – that Benford features improve credit-card fraud detection – from aggregate to per-user and from single-dimensional to multi-dimensional analysis, using the interpretable IEEE-CIS dataset. We developed the mathematical and computational infrastructure for this: a Monte Carlo-derived Excess MAD correction for small-sample per-user analysis, a three-dimensional Benford fingerprint covering transaction amounts, time deltas, and amount ratios, and an end-to-end Random Forest evaluation pipeline.

The result was a negative one, and in three layers. First, at the per-user level, legitimate transactions deviate from Benford’s Law *more* than fraudulent transactions. Second, this inversion holds uniformly across all five product categories. Third, even a Random Forest classifier allowed to use these inverted features freely does not outperform a baseline without them. The mechanisms – subscription-driven price clustering for legitimate users, algorithmic probe distributions for fraudsters, and category-level violation of Hill’s mixture theorem – suggest that the boundary between “Benford detects fraud” and “Benford does not detect fraud” runs between batch financial-statement auditing and real-time retail e-commerce.

Future work could test whether Benford survives at the *merchant* rather than *user* level, where the mixture property may partially recover; whether filtering subscription-like recurring amounts restores the expected signal; or whether second-digit or first-two-digit tests are more sensitive than the first-digit analysis used here.

References

- [1] S. Newcomb. Note on the frequency of use of the different digits in natural numbers. *American Journal of Mathematics*, 4(1):39–40, 1881.
- [2] F. Benford. The law of anomalous numbers. *Proceedings of the American Philosophical Society*, 78(4):551–572, 1938.
- [3] T. P. Hill. A statistical derivation of the significant-digit law. *Statistical Science*, 10(4):354–363, 1995.
- [4] H.-A. Engel and C. Leuenberger. Benford’s Law for exponential random variables. *Statistics & Probability Letters*, 63(4):361–365, 2003.
- [5] C. E. Mungan. Derivation of Benford’s Law. Technical report, U.S. Naval Academy, 2021.
- [6] M. J. Nigrini. *Benford’s Law: Applications for Forensic Accounting, Auditing, and Fraud Detection*. Wiley, 2012.
- [7] C. Durtschi, W. Hillison, and C. Pacini. The effective use of Benford’s Law to assist in detecting fraud in accounting data. *Journal of Forensic Accounting*, 5:17–34, 2004.
- [8] B. R. Fu. Leveraging Benford’s Law and Machine Learning for Financial Fraud Detection. *Cybersecurity Undergraduate Research Showcase*, 7. Old Dominion University Digital Commons, 2025.
- [9] IEEE Computational Intelligence Society and Vesta Corporation. IEEE-CIS Fraud Detection. Kaggle Competition, 2019. <https://www.kaggle.com/competitions/ieee-fraud-detection>

A Code Listings

This appendix contains the core mathematical and algorithmic code used in the analysis. File I/O, argument parsing, and plotting code are omitted for brevity.

A.1 Core Benford Math

```
import numpy as np

def benford_probs():
    """Benford probabilities for digits 1..9:  $P(d) = \log_{10}(1 + 1/d)$ ."""
    digits = np.arange(1, 10)
    return np.log10(1 + 1 / digits)

BENFORD_P = benford_probs()
```

```

def leading_digit(x):
    """Extract the first significant digit using log10 mantissa."""
    x = np.asarray(x, dtype=float)
    x = x[x > 0]
    log_x = np.log10(x)
    mantissa = 10 ** (log_x - np.floor(log_x))
    return np.floor(mantissa).astype(int)

def digit_distribution(values):
    """Return (counts, proportions) of leading digits 1..9."""
    values = np.asarray(values, dtype=float)
    values = values[values > 0]
    if len(values) == 0:
        return np.zeros(9, dtype=int), np.zeros(9, dtype=float)
    digits = leading_digit(values)
    counts = np.bincount(digits, minlength=10)[1:10]
    proportions = counts / counts.sum()
    return counts, proportions

def mad(values):
    """Mean Absolute Deviation from Benford."""
    counts, proportions = digit_distribution(values)
    if counts.sum() < 9:
        return np.nan
    return float(np.mean(np.abs(proportions - BENFORD_P)))

```

A.2 Monte Carlo $E[\text{MAD}]$ Simulation

```

import numpy as np

SAMPLE_SIZES = [10, 15, 20, 25, 30, 40, 50, 75, 100, 150,
                 200, 300, 500, 1000]
N_TRIALS = 5000
RNG = np.random.default_rng(seed=42)

def sample_benford(n):
    return RNG.choice(np.arange(1, 10), size=n, p=BENFORD_P)

def expected_mad(n_samples, n_trials=N_TRIALS):
    """Monte Carlo estimate of  $E[\text{MAD}(N)]$  under Benford null."""
    mads = np.empty(n_trials)
    for i in range(n_trials):
        digits = sample_benford(n_samples)
        counts = np.bincount(digits, minlength=10)[1:10]
        proportions = counts / counts.sum()

```

```

        mads[i] = np.mean(np.abs(proportions - BENFORD_P))
    return mads.mean(), mads.std()

```

Empirical result: $E[MAD] \sim k / \sqrt{N}$, with $k = 0.235$

A.3 Per-User Feature Extraction

```

def expected_mad_for_n(n, emad_table):
    """Look up E[MAD] for sample size n via linear interpolation."""
    if n < emad_table["N"].min():
        return emad_table["E_MAD"].iloc[0]
    if n > emad_table["N"].max():
        n_max = emad_table["N"].max()
        emad_max = emad_table["E_MAD"].iloc[-1]
        return emad_max * np.sqrt(n_max / n)
    return float(np.interp(n, emad_table["N"],
                           emad_table["E_MAD"]))

def excess_mad(values, emad_table):
    """MAD - E[MAD] correction for small-sample bias."""
    values = np.asarray(values, dtype=float)
    values = values[values > 0]
    n = len(values)
    if n < 9:
        return np.nan
    raw_mad = mad(values)
    if np.isnan(raw_mad):
        return np.nan
    return raw_mad - expected_mad_for_n(n, emad_table)

def per_user_features(df, emad_table):
    """For each user with >=30 transactions, compute the 3D
    Benford fingerprint plus baseline transaction features."""
    features = []
    groups = df.sort_values(["user_id", "TransactionDT"]) \
        .groupby("user_id")
    for user_id, g in groups:
        if len(g) < 30:
            continue
        amts = g["TransactionAmt"].values
        time_deltas = np.diff(g["TransactionDT"].values)
        time_deltas = time_deltas[time_deltas > 0]
        mean_amt = amts.mean()
        ratios = amts / mean_amt if mean_amt > 0 else np.array([])
        features.append({

```

```

        "user_id": user_id,
        "n_txn": len(g),
        "mean_amt": mean_amt,
        "std_amt": amts.std(),
        "median_amt": np.median(amts),
        "max_amt": amts.max(),
        "median_time_delta": np.median(time_deltas)
            if len(time_deltas) > 0 else np.nan,
        "benford_amt": excess_mad(amts, emad_table),
        "benford_time": excess_mad(time_deltas, emad_table),
        "benford_ratio": excess_mad(ratios, emad_table),
        "isFraud": int(g["isFraud"].max()),
    })
    return pd.DataFrame(features)

```

A.4 Model Training

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (roc_auc_score, precision_score,
                             recall_score, f1_score)

BASELINE_FEATURES = [
    "n_txn", "mean_amt", "std_amt", "median_amt",
    "max_amt", "median_time_delta",
]
ODU_FEATURES = BASELINE_FEATURES + ["benford_aggregate"]
OURS_FEATURES = BASELINE_FEATURES + [
    "benford_amt", "benford_time", "benford_ratio",
]

def train_and_eval(X_train, X_test, y_train, y_test, name):
    """Train RF and report metrics."""
    clf = RandomForestClassifier(
        n_estimators=200,
        max_depth=10,
        class_weight="balanced",
        random_state=42,
        n_jobs=-1,
    )
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    y_prob = clf.predict_proba(X_test)[:, 1]
    return {
        "model": name,

```

```

        "auc_roc": roc_auc_score(y_test, y_prob),
        "precision": precision_score(y_test, y_pred,
                                     zero_division=0),
        "recall":    recall_score(y_test, y_pred),
        "f1":        f1_score(y_test, y_pred),
    }

idx_train, idx_test = train_test_split(
    np.arange(len(feat_df)),
    test_size=0.25,
    stratify=feat_df["isFraud"].values,
    random_state=42,
)

```